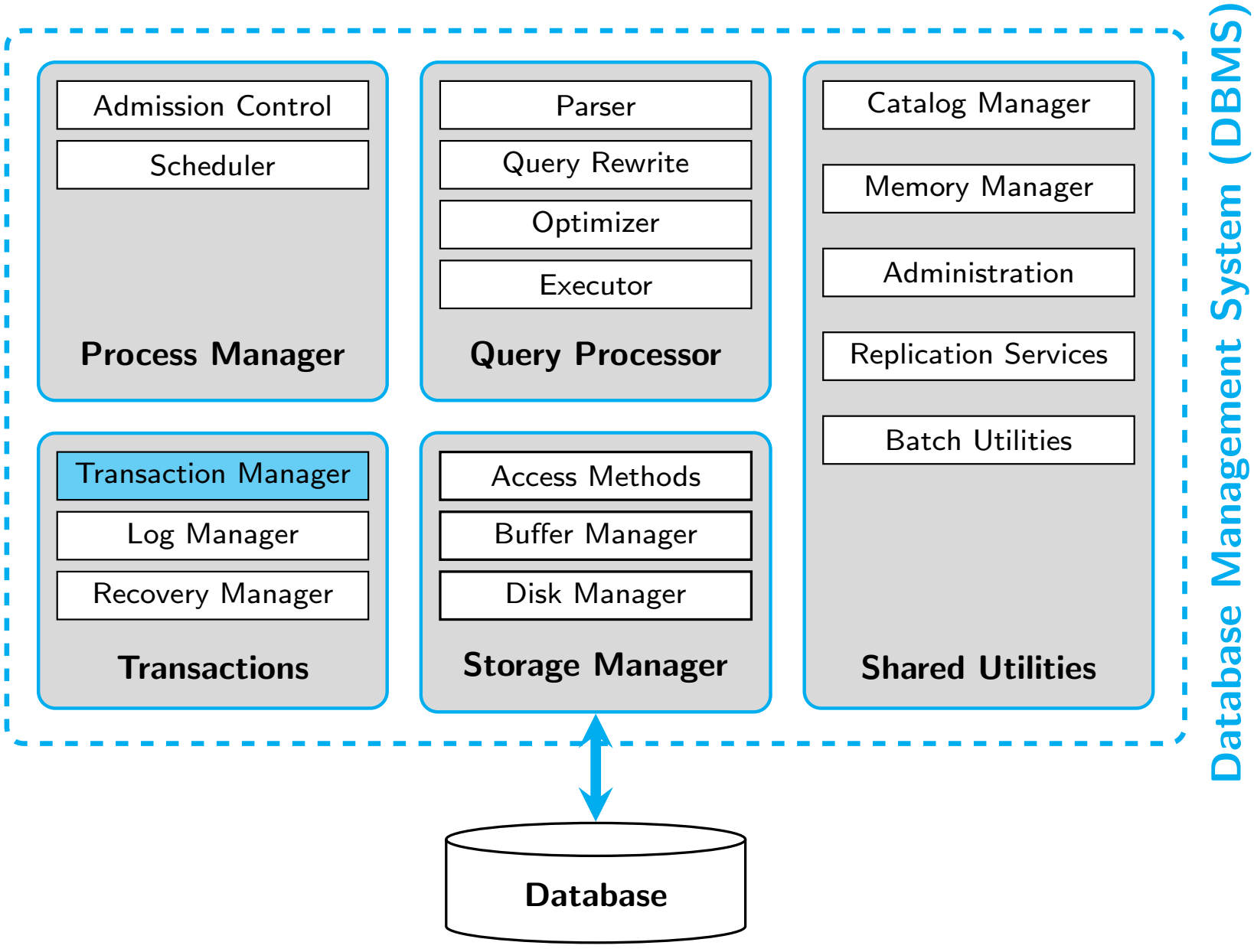
Manejo de transacciones: Parte II

Clase 23

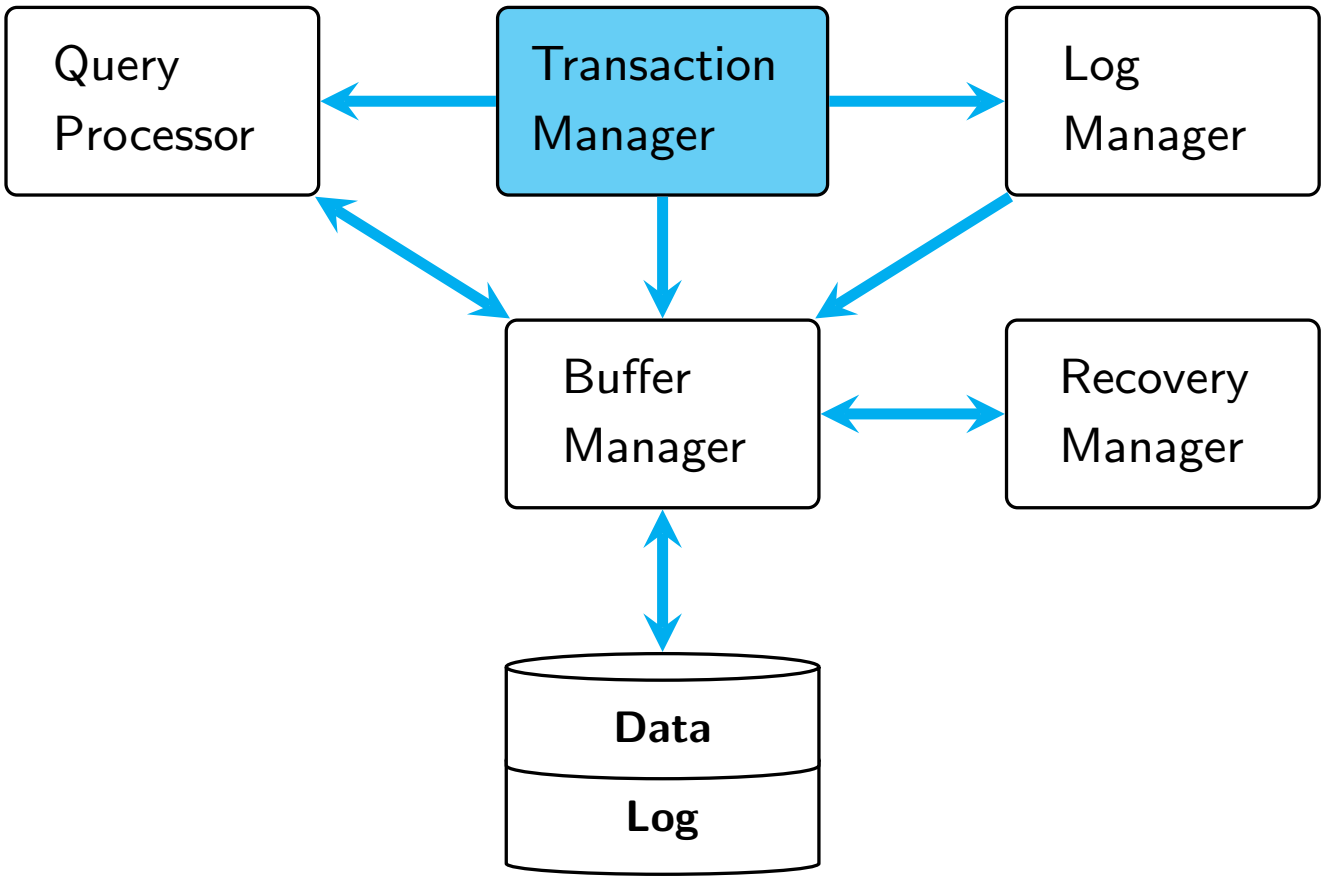
IIC2413 - Sección 3

Prof. Cristian Riveros

Manejo de transacciones



Zoom a la arquitectura de las transacciones



Outline

Transacciones serializables (anterior)

Conflict serializable

Locks

Tuplas fantasmas

Isolation en SQL

Outline

Transacciones serializables (anterior)

Conflict serializable

Locks

Tuplas fantasmas

Isolation en SQL

Schedules, serial y serializable (anterior)

Definición

- Un **schedule** es una secuencia de **instrucciones primitivas** de una o más transacciones $T_1, \ldots, T_n$ tal que cada T_i aparecen en el mismo orden de su definición.
- Un schedule es **serial** si todas las acciones de cada transacción son ejecutas en grupo sin intercalación de acciones.
- Un schedule S es **serializable** si existe algún schedule serial S' tal que el resultado de S y S' es el mismo para todo estado inicial de la BD.

La tarea del Transaccion Manager
es permitir solo schedules que son **serializables**

Schedules, serial y serializable (anterior)

Ejemplo de schedule serial

$\mathbf{T_1}$	$\mathbf{T_2}$	A	B
READ(A, x)		100	50
$x := x + 100$			
WRITE(A, x)		200	
READ(B, x)			
$x := x + 200$			
WRITE(B, x)			250
	READ(A, y)		
	$y := y * 2$		
	WRITE(A, y)	400	
	READ(B, y)		
	$y := y * 3$		
	WRITE(B, y)		750

Schedules, serial y serializable (anterior)

Ejemplo de schedule serializable

$\mathbf{T_1}$	$\mathbf{T_2}$	A	B
READ(A, x)		100	50
$x := x + 100$			
WRITE(A, x)		200	
	READ(A, y)		
	$y := y * 2$		
	WRITE(A, y)	400	
READ(B, x)			
$x := x + 200$			
WRITE(B, x)			250
	READ(B, y)		
	$y := y * 3$		
	WRITE(B, y)		750

Schedules, serial y serializable (anterior)

Ejemplo de schedule no serializable

$\mathbf{T_1}$	$\mathbf{T_2}$	A	B
READ(A, x)		100	50
$x := x + 100$			
WRITE(A, x)		200	
	READ(A, y)		
	$y := y * 2$		
	WRITE(A, y)	400	
	READ(B, y)		
	$y := y * 3$		
	WRITE(B, y)		150
READ(B, x)			
$x := x + 200$			
WRITE(B, x)			350

¿és posible verificar que un schedule es serializable?

Ejemplo de schedule no serializable

$\mathbf{T_1}$	$\mathbf{T_2}$	A	B
READ(A, x)		100	50
$x := x + 100$			
WRITE(A, x)		200	
	READ(A, y)		
	$y := y * 2$		
	WRITE(A, y)	400	
	READ(B, y)		
	$y := y * 3$		
	WRITE(B, y)		150
READ(B, x)			
$x := x + 200$			
WRITE(B, x)			350

¿és posible verificar que un schedule es serializable?

Ejemplo de schedule serializable

T_1	T_2	A	B
READ(A, x)		100	50
$x := x + 100$			
WRITE(A, x)		200	
	READ(A, y)		
	$y := y + 300$		
	WRITE(A, y)	500	
	READ(B, y)		
	$y := y + 400$		
	WRITE(B, y)		450
READ(B, x)			
$x := x + 200$			
WRITE(B, x)			650

¡solo cambiamos dos operaciones!

¿és posible verificar que un schedule es serializable?

Conclusiones

- En general, es muy difícil verificar si un schedule es serializable.
- Simplificamos el análisis
  considerando solo los `READ` y `WRITE` de un objeto.

Simplificación

Desde ahora, las instrucciones de las transacciones
solo tendrán **operaciones** primitivas `READ` y `WRITE` de un objeto:

- $T_i : \texttt{READ}(X, t) \;\Rightarrow\; r_i(X)$
- $T_i : \texttt{WRITE}(X, t) \;\Rightarrow\; w_i(X)$

¿qué tan **razonable** es esta simplificación?

¿és posible verificar que un schedule es serializable?

Ejemplo de simplificación

Antes	**Ahora**
T_1 : READ(A, x)	$r_1(A)$
T_1 : $x := x + 100$	
T_1 : WRITE(A, x)	$w_1(A)$
T_2 : READ(A, y)	$r_2(A)$
T_2 : $y := y * 2$	
T_2 : WRITE(A, y)	$w_2(A)$
T_2 : READ(B, y)	$r_2(B)$
T_2 : $y := y * 3$	
T_2 : WRITE(B, y)	$w_2(B)$
T_1 : READ(B, x)	$r_1(B)$
T_1 : $x := x + 200$	
T_1 : WRITE(B, x)	$w_1(B)$

En línea: $r_1(A)\ w_1(A)\ r_2(A)\ w_2(A)\ r_2(B)\ w_2(B)\ r_1(B)\ w_1(B)$

Outline

Transacciones serializables (anterior)

Conflict serializable

Locks

Tuplas fantasmas

Isolation en SQL

Anomalías de serialización

1. Conflictos Write-Read: lecturas sucias (dirty reads)

Ejemplo

$\mathbf{T_1}$	$\mathbf{T_2}$	A	B	
READ(A, x)		1000	1000	
WRITE$(A, x - 100)$		900		
	READ(A, y)			
	WRITE$(A, y * 1.1)$	990		
	READ(B, y)			
	WRITE$(B, y * 1.1)$		1100	
READ(B, x)				
WRITE$(B, x + 100)$		990	1200	✕

$r_1(A)$ $w_1(A)$ $r_2(A)$ $w_2(A)$ $r_2(B)$ $w_2(B)$ $r_1(B)$ $w_1(B)$

Anomalías de serialización

1. Conflictos Write-Read: lecturas sucias (dirty reads)
2. Conflictos Read-Write: lecturas irrepetibles (unrepeatable read)

Ejemplo

T_1	T_2	A	
	READ(A, y)		1
READ(A, x)			
IF$(x > 0)$			
	IF$(y > 0)$		
	WRITE$(A, y - 1)$	0	
	ENDIF		
WRITE$(A, x - 1)$		-1	
ENDIF		-1	✗

$r_2(A)$ $r_1(A)$ $w_2(A)$ $w_1(A)$

Anomalías de serialización

1. Conflictos Write-Read: lecturas sucias (dirty reads)
2. Conflictos Read-Write: lecturas irrepetibles (unrepeatable read)
3. Conflictos Write-Write: actualización perdida (lost update)

Ejemplo

$\mathbf{T_1}$	$\mathbf{T_2}$	A	B	
WRITE$(A, 1000)$		1000		
	WRITE$(A, 2000)$	2000		
	WRITE$(B, 2000)$		2000	
WRITE$(B, 1000)$		2000	1000	✕

$w_1(A)$ $w_2(A)$ $w_2(B)$ $w_2(B)$

Operaciones conflictivas

Ejemplos anteriores

$$r_1(A)\ \ w_1(A)\ \ r_2(A)\ \ w_2(A)\ \ r_2(B)\ \ w_2(B)\ \ r_1(B)\ \ w_1(B)$$

$$r_2(A)\ \ r_1(A)\ \ w_2(A)\ \ w_1(A)$$

$$w_1(A)\ \ w_2(A)\ \ w_2(B)\ \ w_2(B)$$

Definición

Sea S un schedule de transacciones $T_1, \ldots, T_n$.

Para $T_i \neq T_j$, estas operaciones consecutivas son **conflictivas** en S:

- $w_i(X)\ r_j(X)$
- $r_i(X)\ w_j(X)$
- $w_i(X)\ w_j(X)$

¡estas operaciones no se pueden cambiar!

Operaciones NO conflictivas

Estas operaciones si las podemos cambiar . . .

Para $T_i \neq T_j$, estas operaciones consecutivas **NO** son **conflictivas** en S:

- $r_i(X)\ r_j(Y)$ (lecturas)
- $r_i(X)\ w_j(Y)$ con $X \neq Y$ (escrituras distintas)
- $w_i(X)\ r_j(Y)$ con $X \neq Y$ (escrituras distintas)
- $w_i(X)\ w_j(Y)$ con $X \neq Y$ (escrituras distintas)

Proposición

Si o_1, o_2 son operaciones **NO conflictivas**, entonces
para todo schedules S y S' se tiene:

$$S \cdot o_1 o_2 \cdot S' \quad \text{es equivalente a} \quad S \cdot o_2 o_1 \cdot S'$$

¡siempre podemos **cambiar** transacciones NO conflictivas!

Operaciones NO conflictivas

Proposición

Si o_1, o_2 son operaciones **NO conflictivas**, entonces
para todo schedules S y S' se tiene:

$$S \cdot o_1 o_2 \cdot S' \quad \text{es equivalente a} \quad S \cdot o_2 o_1 \cdot S'$$

¿cómo convertimos este schedule en un schedule serial?

$\mathbf{T_1}$	$\mathbf{T_2}$
$r_1(A)$	
$w_1(A)$	
	$r_2(A)$
	$w_2(A)$
$r_1(B)$	
$w_1(B)$	
	$r_2(B)$
	$w_2(B)$

$$r_1(A)\, w_1(A)\, r_2(A)\, w_2(A)\, r_1(B)\, w_1(B)\, r_2(B)\, w_2(B)$$

Operaciones NO conflictivas

Proposición

Si o_1, o_2 son operaciones **NO conflictivas**, entonces
para todo schedules S y S' se tiene:

$$S \cdot o_1 o_2 \cdot S' \quad \text{es equivalente a} \quad S \cdot o_2 o_1 \cdot S'$$

¿cómo convertimos este schedule en un schedule serial?

$\mathbf{T_1}$	$\mathbf{T_2}$
$r_1(A)$	
$w_1(A)$	
	$r_2(A)$
	$w_2(A)$
$r_1(B)$	
$w_1(B)$	
	$r_2(B)$
	$w_2(B)$

$$\underbrace{r_1(A)\, w_1(A)\, r_2(A)}_{S}\; w_2(A)\, r_1(B)\; \underbrace{w_1(B)\, r_2(B)\, w_2(B)}_{S'}$$

Operaciones NO conflictivas

Proposición

Si o_1, o_2 son operaciones **NO conflictivas**, entonces
para todo schedules S y S' se tiene:

$$S \cdot o_1 o_2 \cdot S' \quad \text{es equivalente a} \quad S \cdot o_2 o_1 \cdot S'$$

¿cómo convertimos este schedule en un schedule serial?

$\mathbf{T_1}$	$\mathbf{T_2}$
$r_1(A)$	
$w_1(A)$	
	$r_2(A)$
$r_1(B)$	
	$w_2(A)$
$w_1(B)$	
	$r_2(B)$
	$w_2(B)$

$$\underbrace{r_1(A)\, w_1(A)\, r_2(A)}_{S}\ r_1(B)\, w_2(A)\ \underbrace{w_1(B)\, r_2(B)\, w_2(B)}_{S'}$$

Operaciones NO conflictivas

Proposición

Si o_1, o_2 son operaciones **NO conflictivas**, entonces
para todo schedules S y S' se tiene:

$$S \cdot o_1 o_2 \cdot S' \quad \text{es equivalente a} \quad S \cdot o_2 o_1 \cdot S'$$

¿cómo convertimos este schedule en un schedule serial?

$\mathbf{T_1}$	$\mathbf{T_2}$
$r_1(A)$	
$w_1(A)$	
	$r_2(A)$
$r_1(B)$	
	$w_2(A)$
$w_1(B)$	
	$r_2(B)$
	$w_2(B)$

$$r_1(A)\, w_1(A)\, r_2(A)\, r_1(B)\, w_2(A)\, w_1(B)\, r_2(B)\, w_2(B)$$

Operaciones NO conflictivas

Proposición

Si o_1, o_2 son operaciones **NO conflictivas**, entonces
para todo schedules S y S' se tiene:

$$S \cdot o_1 o_2 \cdot S' \quad \text{es equivalente a} \quad S \cdot o_2 o_1 \cdot S'$$

¿cómo convertimos este schedule en un schedule serial?

$\mathbf{T_1}$	$\mathbf{T_2}$
$r_1(A)$	
$w_1(A)$	
	$r_2(A)$
$r_1(B)$	
	$w_2(A)$
$w_1(B)$	
	$r_2(B)$
	$w_2(B)$

$$\underbrace{r_1(A)\, w_1(A)}_{S}\; r_2(A)\, r_1(B)\; \underbrace{w_2(A)\, w_1(B)\, r_2(B)\, w_2(B)}_{S'}$$

Operaciones NO conflictivas

Proposición

Si o_1, o_2 son operaciones **NO conflictivas**, entonces
para todo schedules S y S' se tiene:

$$S \cdot o_1 o_2 \cdot S' \quad \text{es equivalente a} \quad S \cdot o_2 o_1 \cdot S'$$

¿cómo convertimos este schedule en un schedule serial?

$\mathbf{T_1}$	$\mathbf{T_2}$
$r_1(A)$	
$w_1(A)$	
$r_1(B)$	
	$r_2(A)$
	$w_2(A)$
$w_1(B)$	
	$r_2(B)$
	$w_2(B)$

$$\underbrace{r_1(A)\, w_1(A)}_{S}\ r_1(B)\, r_2(A)\ \underbrace{w_2(A)\, w_1(B)\, r_2(B)\, w_2(B)}_{S'}$$

Operaciones NO conflictivas

Proposición

Si o_1, o_2 son operaciones **NO conflictivas**, entonces
para todo schedules S y S' se tiene:

$$S \cdot o_1 o_2 \cdot S' \quad \text{es equivalente a} \quad S \cdot o_2 o_1 \cdot S'$$

¿cómo convertimos este schedule en un schedule serial?

$\mathbf{T_1}$	$\mathbf{T_2}$
$r_1(A)$	
$w_1(A)$	
$r_1(B)$	
	$r_2(A)$
	$w_2(A)$
$w_1(B)$	
	$r_2(B)$
	$w_2(B)$

$$r_1(A)\, w_1(A)\, r_1(B)\, r_2(A)\, w_2(A)\, w_1(B)\, r_2(B)\, w_2(B)$$

Operaciones NO conflictivas

Proposición

Si o_1, o_2 son operaciones **NO conflictivas**, entonces
para todo schedules S y S' se tiene:

$$S \cdot o_1 o_2 \cdot S' \quad \text{es equivalente a} \quad S \cdot o_2 o_1 \cdot S'$$

¿cómo convertimos este schedule en un schedule serial?

$\mathbf{T_1}$	$\mathbf{T_2}$
$r_1(A)$	
$w_1(A)$	
$r_1(B)$	
	$r_2(A)$
	$w_2(A)$
$w_1(B)$	
	$r_2(B)$
	$w_2(B)$

$$\underbrace{r_1(A)\, w_1(A)\, r_1(B)\, r_2(A)}_{S}\; w_2(A)\, w_1(B)\; \underbrace{r_2(B)\, w_2(B)}_{S'}$$

Operaciones NO conflictivas

Proposición

Si o_1, o_2 son operaciones **NO conflictivas**, entonces
para todo schedules S y S' se tiene:

$$S \cdot o_1 o_2 \cdot S' \quad \text{es equivalente a} \quad S \cdot o_2 o_1 \cdot S'$$

¿cómo convertimos este schedule en un schedule serial?

$\mathbf{T_1}$	$\mathbf{T_2}$
$r_1(A)$	
$w_1(A)$	
$r_1(B)$	
	$r_2(A)$
$w_1(B)$	
	$w_2(A)$
	$r_2(B)$
	$w_2(B)$

$$\underbrace{r_1(A)\, w_1(A)\, r_1(B)\, r_2(A)}_{S}\; w_1(B)\, w_2(A)\; \underbrace{r_2(B)\, w_2(B)}_{S'}$$

Operaciones NO conflictivas

Proposición

Si o_1, o_2 son operaciones **NO conflictivas**, entonces
para todo schedules S y S' se tiene:

$$S \cdot o_1 o_2 \cdot S' \quad \text{es equivalente a} \quad S \cdot o_2 o_1 \cdot S'$$

¿cómo convertimos este schedule en un schedule serial?

$\mathbf{T_1}$	$\mathbf{T_2}$
$r_1(A)$	
$w_1(A)$	
$r_1(B)$	
	$r_2(A)$
$w_1(B)$	
	$w_2(A)$
	$r_2(B)$
	$w_2(B)$

$$r_1(A)\, w_1(A)\, r_1(B)\, r_2(A)\, w_1(B)\, w_2(A)\, r_2(B)\, w_2(B)$$

Operaciones NO conflictivas

Proposición

Si o_1, o_2 son operaciones **NO conflictivas**, entonces
para todo schedules S y S' se tiene:

$$S \cdot o_1 o_2 \cdot S' \quad \text{es equivalente a} \quad S \cdot o_2 o_1 \cdot S'$$

¿cómo convertimos este schedule en un schedule serial?

$\mathbf{T_1}$	$\mathbf{T_2}$
$r_1(A)$	
$w_1(A)$	
$r_1(B)$	
	$r_2(A)$
$w_1(B)$	
	$w_2(A)$
	$r_2(B)$
	$w_2(B)$

$$\underbrace{r_1(A)\, w_1(A)\, r_1(B)}_{S}\ r_2(A)\, w_1(B)\ \underbrace{w_2(A)\, r_2(B)\, w_2(B)}_{S'}$$

Operaciones NO conflictivas

Proposición

Si o_1, o_2 son operaciones **NO conflictivas**, entonces
para todo schedules S y S' se tiene:

$$S \cdot o_1 o_2 \cdot S' \quad \text{es equivalente a} \quad S \cdot o_2 o_1 \cdot S'$$

¿cómo convertimos este schedule en un schedule serial?

$\mathbf{T_1}$	$\mathbf{T_2}$
$r_1(A)$	
$w_1(A)$	
$r_1(B)$	
$w_1(B)$	
	$r_2(A)$
	$w_2(A)$
	$r_2(B)$
	$w_2(B)$

$$\underbrace{r_1(A)\, w_1(A)\, r_1(B)}_{S}\; w_1(B)\, r_2(A)\; \underbrace{w_2(A)\, r_2(B)\, w_2(B)}_{S'}$$

Operaciones NO conflictivas

Proposición

Si o_1, o_2 son operaciones **NO conflictivas**, entonces
para todo schedules S y S' se tiene:

$$S \cdot o_1 o_2 \cdot S' \quad \text{es equivalente a} \quad S \cdot o_2 o_1 \cdot S'$$

¿cómo convertimos este schedule en un schedule serial?

$\mathbf{T_1}$	$\mathbf{T_2}$
$r_1(A)$	
$w_1(A)$	
$r_1(B)$	
$w_1(B)$	
	$r_2(A)$
	$w_2(A)$
	$r_2(B)$
	$w_2(B)$

$$r_1(A)\, w_1(A)\, r_1(B)\, w_1(B)\, r_2(A)\, w_2(A)\, r_2(B)\, w_2(B)$$

Conflict serializable

Definiciones

- Para operaciones o_1, o_2 **NO conflictivas** y schedules S y S' se define la relación $\vdash$ entre schedules como:

$$S \cdot o_1 o_2 \cdot S' \ \vdash \ S \cdot o_2 o_1 \cdot S'$$

- Para schedules S y S', definimos la relación $S \vdash^* S'$ como la **clausura refleja y transitiva** de $\vdash$ esto es:

$S \vdash^* S'$ si, y solo si, $S = S'$ o existe S'' tal que $S \vdash S''$ y $S'' \vdash^* S'$

$S \vdash^* S'$ si podemos ir en **cero o más pasos** desde S a S' usando $\vdash$

Conflict serializable

Definiciones

- Para operaciones o_1, o_2 **NO conflictivas** y schedules S y S' se define la relación $\vdash$ entre schedules como:

$$S \cdot o_1 o_2 \cdot S' \ \vdash \ S \cdot o_2 o_1 \cdot S'$$

- Para schedules S y S', definimos la relación $S \vdash^* S'$ como la **clausura refleja y transitiva** de $\vdash$ esto es:

$S \vdash^* S'$ si, y solo si, $S = S'$ o existe S'' tal que $S \vdash S''$ y $S'' \vdash^* S'$

- Un schedule S es **conflict serializable** si existe un **schedule serial** S':

$$S \vdash^* S'$$

Es posible llegar a un schedule serial
en cero o más cambios de operaciones **no conflictivas**

Conflict serializable

Definiciones

- Un schedule S es **conflict serializable** si existe un **schedule serial** S':

$$S \vdash^* S'$$

¿cuáles schedules son conflict serializable?

- $r_2(A)\ r_1(B)\ w_1(B)\ w_2(A)$ ✓
- $r_2(A)\ r_1(A)\ w_2(A)\ w_1(A)$ ✗
- $r_1(A)\ w_1(A)\ w_2(A)\ w_2(A)$ ✓

¿cómo **verificamos** si un schedule es conflict serializable?

Grafo de precedencia

Definición

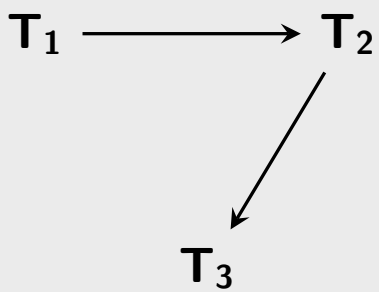
Para un schedule S de $T_1, \ldots, T_n$ se define el **grafo de precedencia**:

$$\mathcal{G}_S \;=\; (\{T_1, \ldots, T_n\}, E_S)$$

tal que $(T_i, T_j) \in E_S$ ssi existen **operaciones conflictivas** $o_i(X)$ y $o'_j(Y)$ tal que $S = S_1 \cdot o_i(X) \cdot S_2 \cdot o_j(Y) \cdot S_3$.

Ejemplo de grafo de precedencia

$\mathbf{T_1}$	$\mathbf{T_2}$	$\mathbf{T_3}$
	$r_2(A)$	
$r_1(B)$		
	$w_2(A)$	
		$r_3(A)$
$w_1(B)$		
		$w_3(A)$
	$r_2(B)$	
	$w_2(B)$	



Grafo de precedencia

Definición

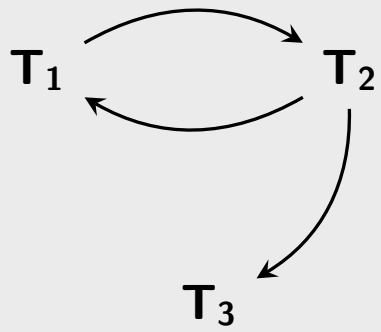
Para un schedule S de $T_1, \ldots, T_n$ se define el **grafo de precedencia**:

$$\mathcal{G}_S \;=\; (\{T_1, \ldots, T_n\}, E_S)$$

tal que $(T_i, T_j) \in E_S$ ssi existen **operaciones conflictivas** $o_i(X)$ y $o'_j(Y)$ tal que $S = S_1 \cdot o_i(X) \cdot S_2 \cdot o_j(Y) \cdot S_3$.

Ejemplo de grafo de precendencia (2)

$\mathbf{T_1}$	$\mathbf{T_2}$	$\mathbf{T_3}$
	$r_2(A)$	
$r_1(B)$		
	$w_2(A)$	
	$r_2(B)$	
		$r_3(A)$
$w_1(B)$		
		$w_3(A)$
	$w_2(B)$	



Conflict serializable y grafo de precedencia

Teorema

Un schedule S es **conflict serializable** si, y solo si,
$\mathcal{G}_S = (\{T_1, \ldots, T_n\}, E_S)$ es **acíclico**.

Demostración (idea)

($\Rightarrow$) Es fácil ver que si $S \vdash^* S'$, entonces $\mathcal{G}_S = \mathcal{G}_{S'}$.
Por último, el grafo de precedencia $\mathcal{G}_S$ de un schedule serial S es acíclico.

($\Leftarrow$) Si $\mathcal{G}_S$ es acíclico, entonces
sea T_1 una **transacción minimal** en el orden parcial formado por E_S.

- Como T_1 no es precedida por ninguna transacción, entonces
  $S \vdash S' \cdot S''$ tal que S' **solo tiene** op. de T_1 y S'' **no tiene** op. de T_1.
- Aplicamos el argumento **inductivamente** sobre $\mathcal{G}_{S''}$ que es acíclico.

□

¿cómo podemos **verificar** esta condición?

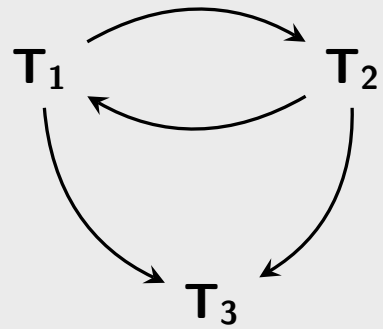
¿conflict serializable $\Leftrightarrow$ serializable?

conflict serializable $\Rightarrow$ serializable

¿es la **otra dirección** también verdadera?

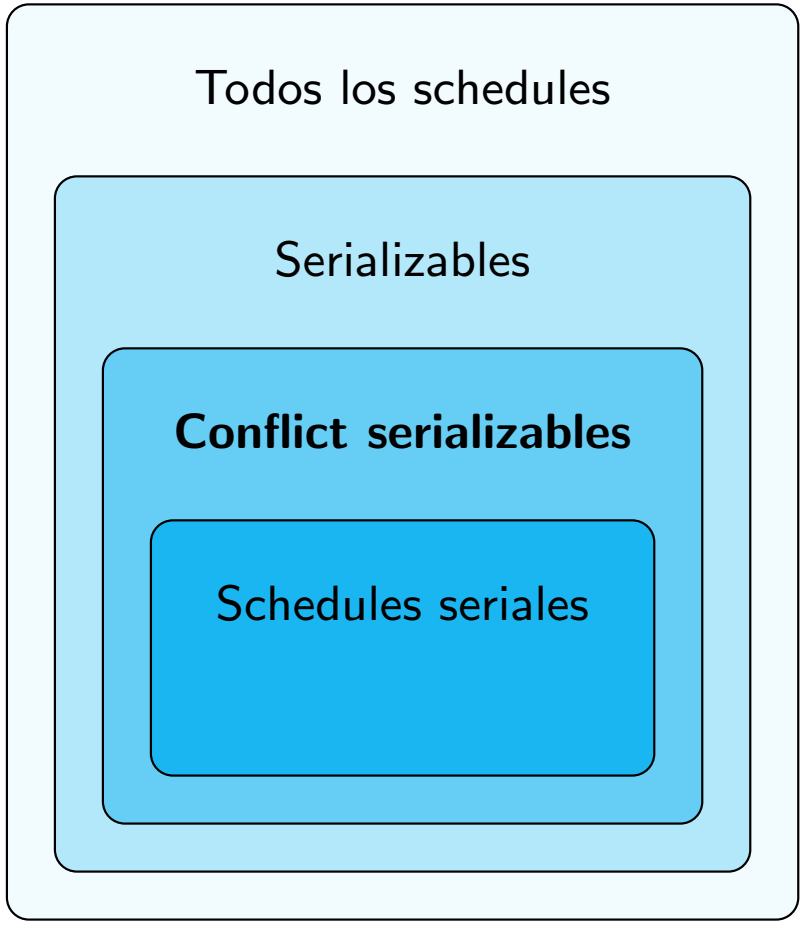
Contraejemplo

$$S = w_1(A)\ w_2(A)\ w_2(B)\ w_1(B)\ w_3(B)$$



- S NO es conflict serializable.
- S es serializable (¿por qué?)

Relación entre tipos de schedules



Conflict serializable es **más fácil de asegurar** que serializable

Outline

Transacciones serializables (anterior)

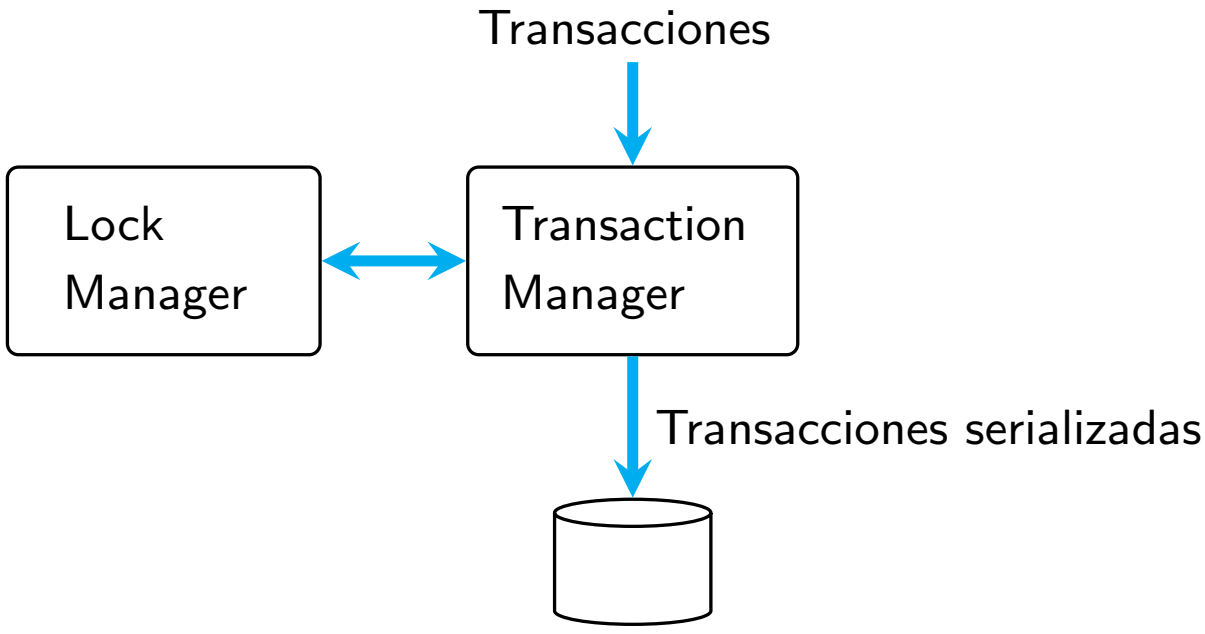
Conflict serializable

Locks

Tuplas fantasmas

Isolation en SQL

Locks



Lock manager (o Lock table) interfaz:

- `requestLock`(*element*).
- `releaseLock`(*element*).

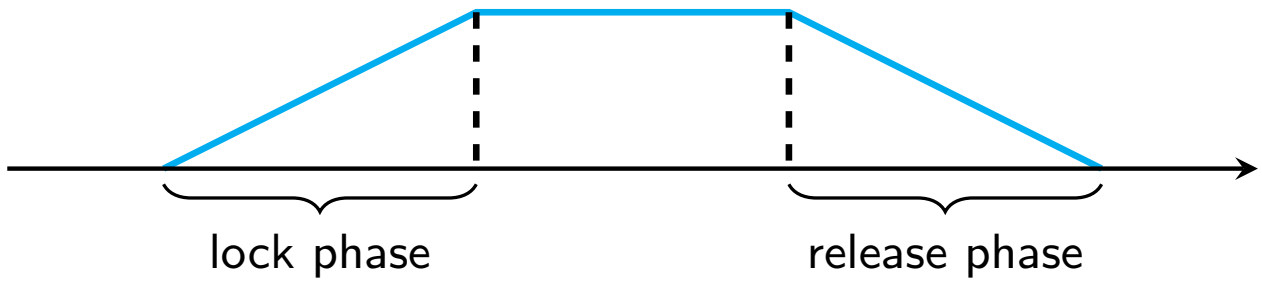
Principios de uso de locks

1. **Consistencia**:
   - Transacciones solo pueden leer o escribir un elemento si tienen su lock respectivo.
   - Si una transacción adquiere un lock, debe hacer released del lock en algún momento.
2. **Validez**:
   - solo una transacción puede tener el lock en cualquier momento.

Two-Phase Locking (2PL)

Política de locks: toda transacción pasa por **dos fases**:

1. **Lock phase**: locks son pedidos a medida que son necesitados.
2. **Release phase**: Locks son liberados si ya no son requeridos.



"Una vez que la transacción libera algún lock,
no puede adquirir ningún lock nuevo."

Two-Phase Locking (2PL)

Ejemplo de ejecución con 2PL

$\mathbf{T_1}$	$\mathbf{T_2}$
$r_1(A)$	$r_2(A)$
$w_1(A)$	$w_2(A)$
$r_1(B)$	$r_2(B)$
$w_1(B)$	$w_2(B)$

$\mathbf{T_1}$	$\mathbf{T_2}$
lock A	
$r_1(A)$	
$w_1(A)$	
lock B	
unlock A	
	lock A
	$r_2(A)$
	$w_2(A)$
$r_1(B)$	
$w_1(B)$	
unlock B	
	lock B
	unlock A
	$r_2(B)$
	$w_2(B)$
	unlock B

Two-Phase Locking (2PL)

Ejemplo de ejecución con 2PL

$\mathbf{T_1}$	$\mathbf{T_2}$
$r_1(A)$	$r_2(A)$
$w_1(A)$	$w_2(A)$
$r_1(B)$	$r_2(B)$
$w_1(B)$	$w_2(B)$

$\mathbf{T_1}$	$\mathbf{T_2}$
$r_1(A)$	
$w_1(A)$	
	$r_2(A)$
	$w_2(A)$
$r_1(B)$	
$w_1(B)$	
	$r_2(B)$
	$w_2(B)$

¡el resultado es un schedule **conflict serializable**!

Two-Phase Locking (2PL)

Teorema

2PL produce solo schedules **conflict serializable**.

Demostración (idea)

Sea S un schedule como resultado del uso de 2PL.

- Para cada transacción T de S,
  sea u_T el tiempo en que T hace el primer unlock de un elemento.
- Ordene las transacciones de S como:

$$T_1, T_2, \ldots, T_n$$

  tal que $u_{T_1} < u_{T_2} < \ldots < u_{T_n}$.

PD: Para cada $i < j$, si $o_i(A)$ hace conflicto con $o_j(A)$ en S,
entonces $o_i(A)$ está antes que $o_j(A)$ en S.

Two-Phase Locking (2PL)

Teorema

2PL produce solo schedules **conflict serializable**.

Demostración (idea)

PD: Para cada $i < j$, si $o_i(A)$ hace conflicto con $o_j(A)$ en S,
entonces $o_i(A)$ está antes que $o_j(A)$ en S.

Por contradicción, suponga que existe una operación $o_i(A)$
que hace conflicto con una operación $o_j(A)$ tal que:

$$S = \ldots\ o_j(A)\ \ldots\ \mathbf{unlock}_j\, A\ \ldots\ \mathbf{lock}_i\, A\ \ldots\ o_i(A)\ \ldots$$

Como $u_{T_i} < u_{T_j}$, entonces el primer $\mathbf{unlock}_i$ de T_i esta antes que $\mathbf{unlock}_j\, A$.

¡contradicción! (¿por qué?)

Por la propiedad, el grafo de precedencia $\mathcal{G}_S$ de S es **acíclico**. □

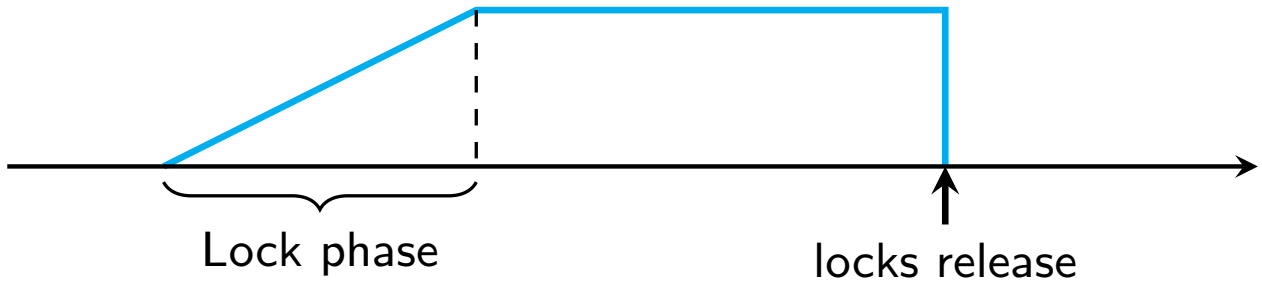
¿cuáles son los problemas con 2PL?

1. Cascading Rollback
2. Interferencia entre operaciones `READ`
3. Deadlocks

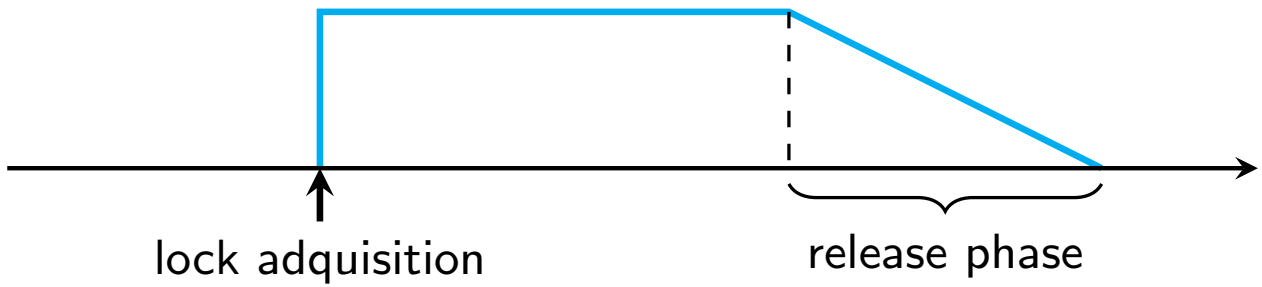
Cascading Rollback

Variaciones de Two-Phase Locking (2PL)

Strict 2PL.



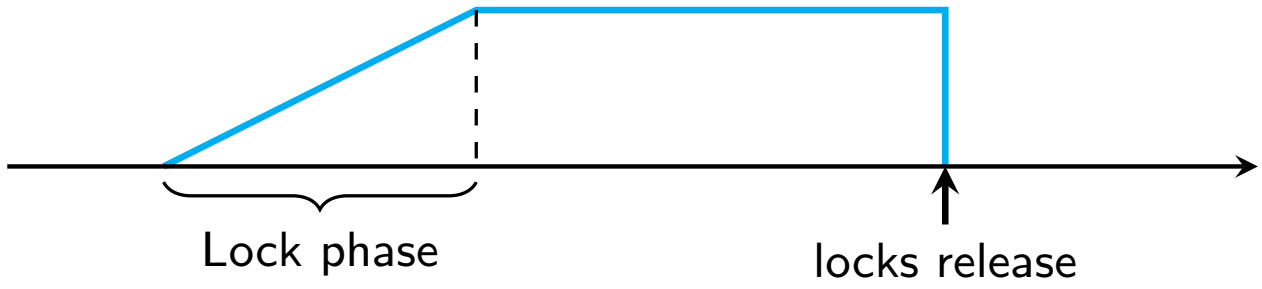
Preclaiming 2PL.



Cascading Rollback

Variaciones de Two-Phase Locking (2PL)

Strict 2PL.



Ventaja de Strict 2PL

- **NO** necesita de **Cascading Rollback**.
- Mas sencillo de implementar.

¿cuáles son los problemas con 2PL?

1. Cascading Rollback
   - Solución: Strict 2PL
2. Interferencia entre operaciones `READ`
3. Deadlocks

Interferencia entre operaciones `READ`

Tipos de lock modes

1. **Shared** locks (slock): para leer.
2. **Exclusive** locks (xlock): para escribir.

Reglas

- **slocks** pueden ser compartidos por varias transacciones.
- Una vez solicitado un **xlock** X de E, X debe esperar la liberación de todos los **slocks** o **xlocks** de E antes de ser otorgado.
- Una vez otorgado un **xlock** X de E, ningún **slock** o **xlock** de E puede ser otorgado hasta que el **xlock** X de E sea liberado.

Interferencia entre operaciones `READ`

Tipos de lock modes

1. **Shared** locks (slock): para leer.
2. **Exclusive** locks (xlock): para escribir.

Reglas

		Lock solicitado	
		S	X
Locks	S	Si	No
Actuales	X	No	No

¿cuáles son las **ventajas** de estos modos de locks?

Interferencia entre operaciones `READ`

Tipos de lock modes

1. **Shared** locks (slock): para leer.
2. **Exclusive** locks (xlock): para escribir.

Reglas

		Lock solicitado	
		S	X
Locks	S	Si	No
Actuales	X	No	No

Teorema

2PL + lock modes produce solo **conflict** serializable schedules.

¿cuáles son los problemas con 2PL?

1. Cascading Rollback
   - Solución: Strict 2PL
2. Interferencia entre operaciones `READ`
   - Solución: Shared y exclusive locks
3. Deadlocks



Deadlocks

Alternativas para evitar deadlocks

- Detección de ciclos en grafo de espera
- Timeout
- Prevención de deadlocks (timestamps):
  - wait-die
  - wound-wait

¿cuáles son los problemas con 2PL?

1. Cascading Rollback
   - Solución: Strict 2PL
2. Interferencia entre operaciones `READ`
   - Solución: Shared y exclusive locks
3. Deadlocks
   - Solución: detección/prevención de deadlocks

Outline

Transacciones serializables (anterior)

Conflict serializable

Locks

Tuplas fantasmas

Isolation en SQL

Tuplas fantasmas / the phantom problem



Suposición

"Transacciones solo modifican: ¡no hay inserts o deletes!"

¿qué cambia si hacemos **inserts** o **deletes**?

Tuplas fantasmas / the phantom problem

Contraejemplo

	$\mathbf{T_1}$	$\mathbf{T_2}$	
Suma de valores $D_1, \ldots, D_n$ en y	$\texttt{READ}(D_1, x)$		
	$y := y + x$		
	$\vdots$		
	$\texttt{READ}(D_n, x)$		
	$y := y + x$		
		$\texttt{INSERT}(D_{n+1}, b)$	Insertar b y escribir en X
		$\texttt{WRITE}(X, c)$	
Escribir y en Y	$\texttt{WRITE}(Y, y)$		
Escribir en X	$\texttt{WRITE}(X, a)$		

¡este schedule **NO es serializable**!

Solución a tuplas fantasmas



Locks de mayor granularidad

- Atributo de una tupla
- Tupla/record
- Página
- Relación

Locking jerárquico (usado por DBMS comerciales)

Outline

Transacciones serializables (anterior)

Conflict serializable

Locks

Tuplas fantasmas

Isolation en SQL

Anomalías de serialización (recordatorio)

1. Conflictos Write-Read: lecturas sucias (dirty reads)
2. Conflictos Read-Write: lecturas irrepetibles (unrepeatable read)
3. Conflictos Write-Write: actualización perdida (lost update)

Niveles de Isolation en SQL

```
SET TRANSACTION ISOLATION LEVEL ****
```

1. SERIALIZABLE
2. REPEATABLE READ
3. READ COMMITTED
4. READ UNCOMMITTED

Niveles de Isolation en SQL

Nivel de Isolation	lecturas sucias	lecturas irrepetibles	actualización perdida	tuplas fantasmas
`SERIALIZABLE`	✗	✗	✗	✗
`REPEATABLE READ`	✗	✗	✗	✓
`READ COMMITTED`	✗	✓	✓	✓
`READ UNCOMMITTED`	✓	✓	✓	✓

1. `SERIALIZABLE`: Strict 2PL y locks jerárquicos.
2. `REPEATABLE READ`: Strict 2PL y locks en tuplas.
3. `READ COMMITTED`: Uso de slocks, pero son liberados inmediatamente.
4. `READ UNCOMMITTED`: Uso de xlocks pero no de slocks.

Utilidad de distinto nivel

1. `SERIALIZABLE`
   - Mayor nivel de isolation, menor performance.
2. `REPEATABLE READ`
   - Útil para transacciones sin `INSERT`/`DELETE`.
3. `READ COMMITTED`
   - Útil para transacciones con pocas modificaciones.
4. `READ UNCOMMITTED`
   - Útil para transacciones de solo lectura o tolerante a errores.

¿qué usan los sistemas clásicos?

Sistema	**Nivel por defecto**	**Nivel máximo**
Postgres	`READ COMMITTED`	`SERIALIZABLE`
MySQL	`REPEATABLE READ`	`SERIALIZABLE`
IBM DB2	`CURSOR STABILITY`	`REPEATABLE READ`
Oracle	`READ COMMITTED`	`SERIALIZABLE`
SQL Server	`READ COMMITTED`	`SERIALIZABLE`

(en DB2, `REPEATABLE READ` es equivalente a `SERIALIZABLE`)

OJO: los sistemas NO usan el máximo estándar **por default**